

BFS

```
import java.util.*;

public class BFS {

    public static void bfs(int start, ArrayList<ArrayList<Integer>> graph, int V) {
        boolean[] visited = new boolean[V];
        Queue<Integer> queue = new LinkedList<>();

        visited[start] = true;
        queue.add(start);

        System.out.print("BFS Traversal: ");

        while (!queue.isEmpty()) {
            int node = queue.poll();
            System.out.print(node + " ");

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.add(neighbor);
                }
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number of vertices: ");
int V = sc.nextInt();

System.out.print("Enter number of edges: ");
int E = sc.nextInt();

ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

for (int i = 0; i < V; i++) {
    graph.add(new ArrayList<>());
}

System.out.println("Enter edges (u v):");
for (int i = 0; i < E; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();

    graph.get(u).add(v);
    graph.get(v).add(u); // undirected graph
}

System.out.print("Enter starting node: ");
int start = sc.nextInt();

bfs(start, graph, V);
}
}
```

Output

Enter number of vertices: 6

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

2 4

3 5

Enter starting node: 0

BFS Traversal: 0 1 2 3 4 5

DFS

```
import java.util.*;

public class DFS {

    public static void dfs(int node, ArrayList<ArrayList<Integer>> graph, boolean[] visited) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, graph, visited);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int V = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int E = sc.nextInt();

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
```

```

for (int i = 0; i < V; i++) {
    graph.add(new ArrayList<>());
}

System.out.println("Enter edges (u v):");
for (int i = 0; i < E; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();

    graph.get(u).add(v);
    graph.get(v).add(u); // undirected graph
}

boolean[] visited = new boolean[V];

System.out.print("Enter starting node: ");
int start = sc.nextInt();

System.out.print("DFS Traversal: ");
dfs(start, graph, visited);
}
}

```

Output

Enter number of vertices: 6

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

2 4

3 5

Enter starting node: 0

DFS Traversal: 0 1 3 5 2 4